

AI WeatherWise – Project Documentation

Intelligent weather forecasts and insights



AI WeatherWise – Intelligent Weather Forecasts and Insights

Project Documentation

AI WeatherWise is a secure, modular RESTful backend platform that delivers real-time weather metrics, natural-language summaries, and personalized activity and clothing recommendations. The system combines Node.js, Express.js, MongoDB, Mongoose, and Google Gemini AI to transform live environmental data into useful, context-aware guidance.

This document presents the project through eight completed phases: Brainstorming & Ideation, Requirement Analysis, Project Design, Project Planning, Project Development, Project Testing, Project Documentation, and Project Demonstration.

Project objective

Provide reliable weather information and intelligent recommendations through maintainable REST APIs, secure authentication, favorite-location management, public access routes, and resilient fallback operation.

1. Brainstorming & Ideation Phase

Initial Idea

The initial idea was to build a secure RESTful backend that provides real-time weather metrics, natural-language weather summaries, and personalized activity and clothing recommendations through Google Gemini AI. The platform would combine dependable weather retrieval with an intelligent interpretation layer so users could make practical decisions without studying raw measurements.

Target User and Scenario

Alex is a travel enthusiast who frequently visits different cities. Alex needs a streamlined way to save favorite locations, track local weather conditions, and receive useful recommendations for outdoor activities and suitable clothing based on current forecasts.

Identified Problems

- **Location management:** Tracking and managing several favorite weather locations is inconvenient.
- **Personalization:** Standard weather data does not provide tailored activity or clothing recommendations.
- **Interpretation:** Raw weather metrics can be difficult to understand quickly.
- **Synchronization:** Users need a secure way to store and synchronize preferences across devices.

Proposed Solution

AI WeatherWise is a centralized weather and AI insight service with secure authentication, favorite-location management, real-time weather retrieval, and Gemini-powered summaries. It includes resilient fallback operation when external API keys are unavailable and supports both authenticated users and public users through appropriate API access routes.

2. Requirement Analysis Phase

Functional and Technical Requirements

Requirement category	Defined requirements
Functional	User registration and login; JWT authentication; bcrypt password hashing; role-based authorization; CRUD operations for favorite locations; real-time weather retrieval; Gemini-powered summaries and recommendations; public weather endpoints; API health monitoring; centralized error handling; request sanitization; and fallback mode when external API keys are unavailable.
Software and platform	Windows 10/11, macOS, or Linux; Node.js v16 or above; npm v8 or above; Express.js; MongoDB; Mongoose; @google/genai; Postman; and Visual Studio Code or another suitable IDE.
Hardware	Intel Core i5 8th Gen or AMD Ryzen 5 equivalent; 8 GB RAM minimum, with 16 GB recommended for concurrent services; and 1 GB of available storage.

Roles and Access Expectations

- **Admin:** Manages user accounts, monitors API health and system logs, and oversees database integrity and protected administrative operations.
- **Registered User:** Registers and authenticates securely, creates and maintains favorite locations, requests weather data and AI insights, and manages personal account details.
- **Weather Application User:** Uses public weather endpoints to consume current metrics without requiring an account, subject to the access rules of each endpoint.

Requirement principle

The platform must remain useful and safe even when optional external integrations are unavailable. Validation, fallback behavior, and centralized error handling are therefore treated as core requirements rather than optional enhancements.

3. Project Design Phase

MVC Architecture

AI WeatherWise follows the Model–View–Controller pattern to separate data management, application processing, and the external interface.

- **Model layer:** User and Location Mongoose schemas define data structure, required fields, relationships, defaults, and persistence rules for MongoDB.
- **Controller layer:** Authentication, validation, weather retrieval, AI orchestration, location management, and standardized JSON responses are handled by focused controllers.
- **View or interface layer:** Because the system is a headless backend, API routing serves as the View layer by connecting HTTP requests to controller logic rather than rendering web pages.

System Components

- Express server gateway for ports, HTTP methods, JSON payloads, CORS, and route registration.
- Authentication and role middleware for JWT validation and permission checks.
- Granular service modules for weather retrieval, AI orchestration, and domain rules.
- Database interface layer using Mongoose to translate application operations into MongoDB queries.
- AI service integration using Google Gemini for summaries and personalized recommendations.
- CORS and JSON middleware, centralized error handling, request validation, and sanitization safeguards.

Entity–Relationship Design

Entity	Fields	Relationship
User	_id; name; email; password; role	One User can have many Locations.
Location	_id; user foreign key referencing User; city; country; createdAt	Each Location belongs to one User.

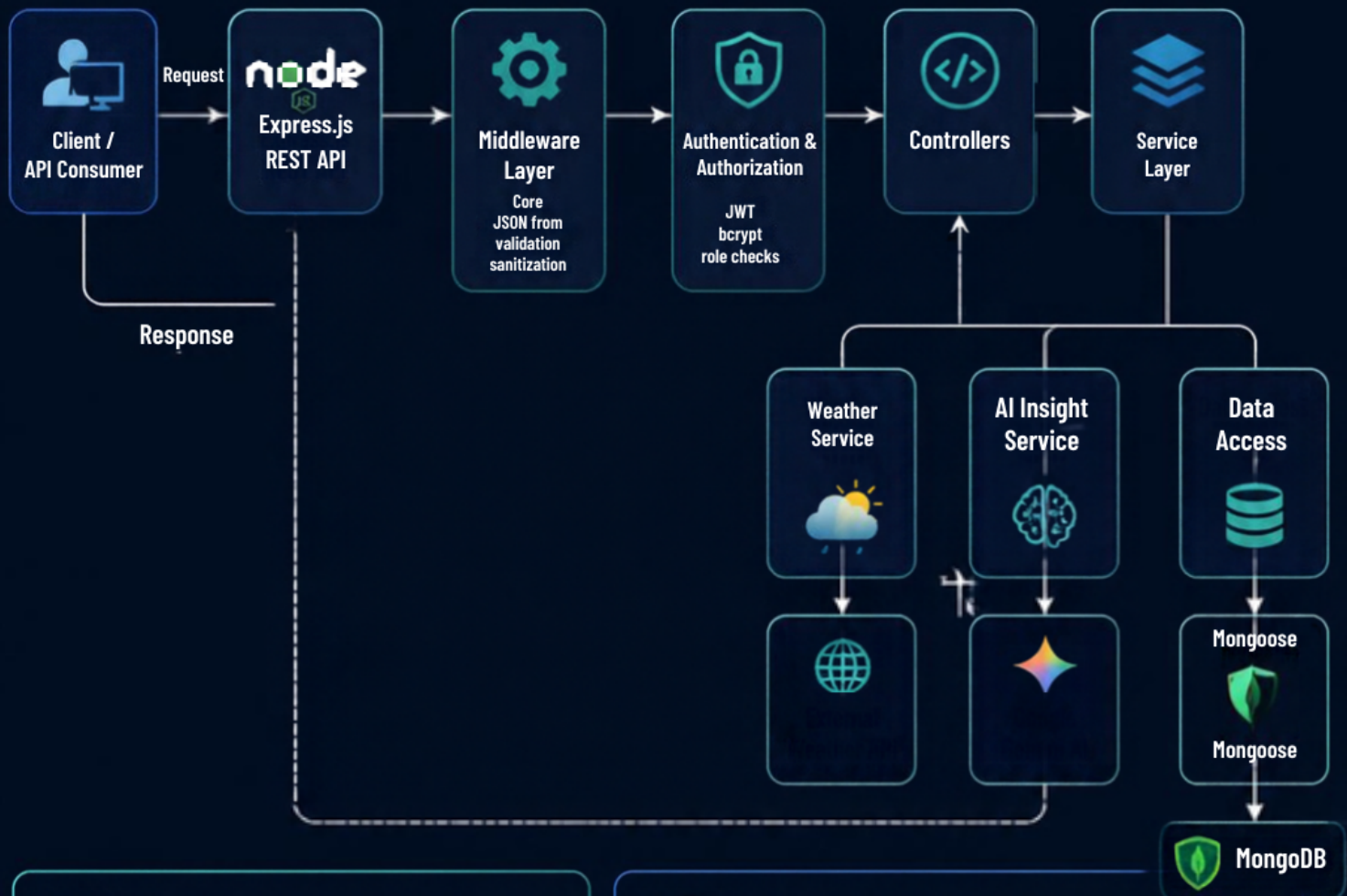
User Flow

1. The user registers or logs in.
2. The authenticated user saves a favorite city through the API.

3. The user fetches current weather metrics for that city.
4. The user requests an AI summary or recommendation.
5. The system validates the JWT and returns the requested response.



AI WeatherWise – Project Architecture



Reliability and Security



Centralized error handling for consistent and user-friendly error responses.



Fallback mode when external API keys are unavailable.



Protected environment variables for sensitive configuration.



Ownership checks for favorite locations to ensure data access control.

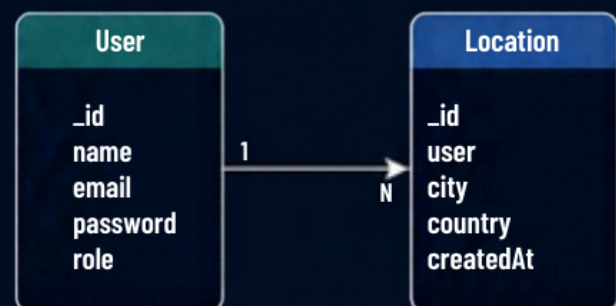


Public versus authenticated routes to separate open access and protected endpoints.



Core Entities

User 1-to-many Location



Request Flow (Left to Right) | Response Flow (Right to Left)



Solid lines represent the request path from client to services. Dashed lines represent the response path back to the client.



4. Project Planning Phase

Implementation Plan

Stage	Planned activities	Deliverable
1	Create the AI WeatherWise project folder and initialize Node.js.	Working project workspace and package manifest.
2	Install dependencies and configure environment variables.	Configured development environment without exposed secrets.
3	Design User and Location schemas and establish the database connection.	Validated MongoDB persistence layer.
4	Implement registration, login, JWT authentication, password hashing, and authorization.	Protected identity and role-management workflows.
5	Implement favorite-location CRUD operations and weather endpoints.	Usable authenticated and public weather APIs.
6	Integrate Gemini summaries and recommendations, including fallback behavior.	AI insight service with resilient operation.
7	Test APIs with Postman or Thunder Client.	Verified endpoint, validation, security, and error scenarios.
8	Document, demonstrate, and review the complete system.	Final documentation, demonstration evidence, and review findings.

Planned Folder and File Structure

- index.js, .env, package.json, and .gitignore at the project root.
- routes for API route definitions.
- controllers for request processing and JSON responses.
- models for User and Location schemas.

- middleware for JWT validation, roles, sanitization, and errors.
- config for database and application configuration.
- services for weather retrieval and Gemini AI integration.

Environment Configuration

- PORT=5000
- MONGO_URI=your_mongodb_connection_string
- JWT_SECRET=your_jwt_secret
- GEMINI_API_KEY=your_gemini_api_key

Responsibility allocation

Admin: User-account management, API-health oversight, system-log monitoring, and database-integrity supervision.

Registered User: Favorite-location management, weather and AI requests, and personal credential maintenance.

5. Project Development Phase

Technology Stack and Setup

The backend is implemented with Node.js, Express.js, MongoDB, and Mongoose using a modular architecture. The separation of routes, controllers, middleware, models, configuration, and services supports maintainability, testing, and future scalability.

Setup Commands

1. `npm init -y`
2. `npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai`
3. `npm install --save-dev nodemon`

Application Entry Point

The `index.js` file loads environment variables, creates the Express server, configures JSON and CORS middleware, establishes the MongoDB connection, registers API routes, and starts the server only after a successful database connection.

Implemented Modules

- **Data layer:** User and Location models enforce required fields, password storage rules, ownership references, and creation timestamps.

- **Authentication:** Controllers register users, hash passwords with bcrypt, verify credentials, and issue JWTs for authenticated sessions.
- **Authorization:** JWT middleware validates tokens, while role checks protect admin-only operations.
- **Location management:** CRUD controllers allow registered users to create, view, update, and delete their own favorite locations.
- **Weather and AI services:** Weather integration retrieves current metrics, while the Gemini recommendation service produces summaries, activity suggestions, and clothing guidance.
- **Public access:** Public weather routes provide current metrics without requiring an account where permitted by the API design.
- **Reliability and security:** Validation, sanitization, CORS configuration, centralized errors, and fallback mode protect the service and preserve useful operation when external keys are unavailable.

6. Project Testing Phase

Testing Strategy

Testing covers unit-level validation, integration behavior, authentication, authorization, database persistence, API endpoints, error handling, fallback mode, and security controls. APIs are tested using Postman or Thunder Client. Successful responses, validation errors, unauthorized responses, and service-failure behavior should be verified without inventing results that have not been observed.

Test ID	Test Scenario	Expected Result	Status
TC-01	User registration with valid details	User is created and a success response is returned without exposing the password.	To be verified during execution
TC-02	Login with valid credentials	Credentials are accepted and a JWT is issued.	To be verified during execution
TC-03	Login with invalid credentials	Authentication is rejected with an appropriate error response.	To be verified during execution
TC-04	Access a protected route without a JWT	Request is denied as unauthorized.	To be verified during execution
TC-05	Save a favorite location	Valid city and country data are stored for the authenticated user.	To be verified during execution
TC-06	Retrieve current weather	Current weather metrics are returned for a valid city.	To be verified during execution
TC-07	Request an AI recommendation	Gemini returns a weather summary or recommendation when configured.	To be verified during execution
TC-08	Submit invalid city input	Validation rejects the input and returns a clear client error.	To be verified during execution
TC-09	Simulate MongoDB	Application reports	To be verified

	connection failure	the failure through controlled error handling.	during execution
TC-10	Run without a Gemini API key	Fallback behavior provides a safe alternative response or controlled limitation.	To be verified during execution
TC-11	Access an admin-only route as a non-admin	Request is denied by role-based authorization.	To be verified during execution

Verification Areas

- **Unit-level validation:** Check required fields, input formats, ownership rules, and sanitization behavior.
- **Integration testing:** Confirm that routes, controllers, services, MongoDB, and external weather or AI integrations work together.
- **Security testing:** Confirm password hashing, JWT validation, role restrictions, protected secrets, CORS behavior, and safe error messages.

7. Project Documentation Phase

Documentation Coverage

The completed documentation should provide a consistent reference for users, developers, administrators, testers, and reviewers. It includes the following sections:

- Project overview and purpose.
- Scenario-based case study describing Alex, the identified problems, and the proposed solution.
- Software and hardware requirements.
- MVC architecture, system components, and database design.
- API descriptions for authentication, locations, weather, AI insights, health monitoring, and public access.
- Setup instructions, environment configuration, execution instructions, and testing procedure.
- Security measures, user roles, limitations, and future enhancements.

Execution Instructions

MongoDB must be running and the required environment variables must be configured before execution. From the project directory, install dependencies and start the server:

1. npm install
2. npm start

After successful execution, the application should connect to MongoDB and begin listening for API requests. Endpoint verification should be performed using Postman or Thunder Client.

Security Measures

- Passwords are hashed before storage.
- JWT validation protects authenticated routes.
- Role-based access control protects administrative operations.
- Input sanitization reduces XSS and NoSQL parameter-manipulation risks.
- Centralized error handling provides controlled responses.
- Secrets remain in environment variables rather than source files.
- CORS configuration controls cross-origin access according to deployment needs.

Limitations and Future Enhancements

Current limitations may include dependence on external weather and AI services, incomplete live-result verification until execution, and the absence of a dedicated visual dashboard. Future enhancements include weather caching, notifications, dashboards, forecast history, multilingual recommendations, and deployment monitoring.

8. Project Demonstration Phase

Demonstration Workflow

1. Start MongoDB and confirm that the database service is available.
2. Configure the .env file with PORT=5000, MONGO_URI, JWT_SECRET, and GEMINI_API_KEY placeholders or valid deployment values.
3. Run npm install to install the project dependencies.
4. Run npm start and confirm that the server starts after the database connection succeeds.
5. Verify the health endpoint and observe the API status response.
6. Register a user with valid account details.
7. Log in and obtain a JWT for protected requests.
8. Create a favorite location using the authenticated request.
9. Fetch current weather for the selected city.
10. Request an AI weather summary or activity and clothing recommendation.
11. Test a public weather endpoint without an account where supported.

12. Demonstrate invalid-token rejection and fallback behavior when the Gemini key is unavailable.

Demonstration References

- **API testing video:** API testing.mp4
- **Backend demonstration video:** Backend demo.mp4
- **CodeDrive link:** Placeholder for the supplied CodeDrive reference.

Final Outcome

AI WeatherWise reduces the effort required to understand raw weather data, securely stores users' favorite locations, and provides context-aware recommendations through Gemini AI. Its modular REST APIs, role-aware security, validation, fallback behavior, and centralized error handling provide a maintainable foundation for future expansion.

Conclusion

The project demonstrates how a secure backend can combine real-time environmental data with artificial intelligence to create practical user value. Through the eight completed phases, AI WeatherWise establishes a clear path from ideation and requirements to implementation, verification, documentation, and demonstration.

References and Placeholders

- API testing.mp4 — supplied API testing demonstration reference.
- Backend demo.mp4 — supplied backend demonstration reference.
- CodeDrive — supplied project repository reference; link to be inserted when available.